CSL 100/ MAL 100

Tutorial 7

Recursive function and Array

1) Find All Possible Permutations of an Array

Write a C program to generate all possible permutations of an array using recursion. Your program should take an array of n distinct integers and print each permutation in a separate line.

Input: arr = {1, 2, 3}

Output:

1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

void permute(int arr[], int l, int r) {
    if (l == r) {
        for (int i = 0; i <= r; i++)
            printf("%d ", arr[i]);
        printf("\n");
    } else {
        for (int i = l; i <= r; i++) {
            swap(&arr[l], &arr[i]);
            permute(arr, l + 1, r);
            swap(&arr[l], &arr[i]);  // backtrack
        }
    }
}
```

```c
int main() {
    int arr[] = {1, 2, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Permutations:\n");
    permute(arr, 0, n - 1);
    return 0;
}
```

2) Sum of All Subsets of an Array

Given an array, find the sum of all its subsets using recursion. The function should return the total sum of all possible subsets.

Input: Array: [3, 1, 2]

Output: Sum of All Subsets: 24

Explanation: The subsets are: {}, {3}, {1}, {2}, {3, 1}, {3, 2}, {1, 2}, and {3, 1, 2}. The sum is: 0 + 3 + 1 + 2 + 4 + 5 + 3 + 6 = 24.

```
#include <stdio.h>

int sumOfSubsets(int arr[], int n, int index, int currSum) {
    if (index == n) {
        return currSum;
    }
    // Sum without including the current element
    int exclude = sumOfSubsets(arr, n, index + 1, currSum);

    // Sum including the current element
    int include = sumOfSubsets(arr, n, index + 1, currSum +
arr[index]);

    return exclude + include;
}
int main() {
    int arr[] = {3, 1, 2};
    int n = sizeof(arr) / sizeof(arr[0]);

    int totalSum = sumOfSubsets(arr, n, 0, 0);
    printf("Sum of All Subsets: %d\n", totalSum);
    return 0;
}
```

Output should be: The subsets are: {}, {3}, {1}, {2}, {3, 1}, {3, 2}, {1, 2}, and {3, 1, 2}. The sum is: 0 + 3 + 1 + 2 + 4 + 5 + 3 + 6 = 24.

3) Generate Pascal's Triangle

Write a recursive function to generate the first n rows of Pascal's triangle, where each number is the sum of the two numbers directly above it. This triangle is named after Blaise Pascal and has many applications in algebra and combinatorics.

Input: Number of rows: n = 5

Output:

```
Pascal's Triangle:
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
```

```c
#include <stdio.h>

int binomialCoefficient(int n, int k) {
    if (k == 0 || k == n) return 1;
    return binomialCoefficient(n - 1, k - 1) +
binomialCoefficient(n - 1, k);
}
void printPascal(int n, int row) {
    if (row == n) return; // Base case

    // Print leading spaces for center alignment
    for (int i = 0; i < n - row - 1; i++) {
        printf(" ");
    }
    for (int col = 0; col <= row; col++) {
        printf("%d ", binomialCoefficient(row, col));    }
    printf("\n");

    printPascal(n, row + 1); // Recur for the next row
}
```

```c
int main() {
    int n = 5; // Number of rows in Pascal's Triangle
    printf("Pascal's Triangle:\n");
    printPascal(n, 0); // Start from row 0
    return 0;
}
```

Question

Minimum Number of Jumps to Reach the End of an Array

- **Problem Statement:** Given an array where each element represents the maximum number of steps you can jump forward, find the minimum number of jumps required to reach the end of the array. Use recursion to solve the problem.

Input: Array: [2, 3, 1, 1, 2, 4, 2, 0, 1, 1]

Output: Minimum Jumps: 4

Explanation of the Output: The minimum jumps required to reach the end are 4: 2 -> 3 -> 2 -> 4 -> End

Let's break down how to reach the end of the array (the last index) effectively:

Start at Index 0: The value is 2, meaning you can jump to either index 1 or index 2. Jump to Index 1 (**first jump**): From index 1, the value is 3, so you can jump to indices 2, 3, or 4. Jump to Index 4 (**second jump**): From index 4, the value is 2, allowing you to jump to indices 5 or 6. Jump to Index 5 (**third jump**): From index 5, the value is 4, allowing you to jump to indices 6, 7, 8, or 9. Finally, Jump to Index 9 (**fourth jump**): Index 9 is the last index of the array, and you have successfully reached the end.

Jumps Taken: From Index 0 to Index 1 → First Jump From Index 1 to Index 4 → Second Jump From Index 4 to Index 5 → Third Jump From Index 5 to Index 9 → Fourth Jump Based on this path, the minimum number of jumps required to reach the end of the array is 4.